专业术语翻译对照表

按原文涉及的概念整理。正文采用以下译名，并以括号标注英文原词。模型、库、方法和 API 标识保留原名。

英文原词	中文译名	缩写／符号
Attention	注意力机制	Attention
Query	查询	Q
Key	键	K
Value	值	V
Key-value Pair	键值对	KV
Transformer	Transformer 模型	Transformer
Large Language Model	大语言模型	LLM
Scaled Dot-product Attention	缩放点积注意力	SDPA
Dot-product Attention	点积注意力	
Additive Attention	加性注意力	
Self-attention	自注意力	
Cross-attention	交叉注意力	
Causal Attention	因果注意力	
Causal Self-attention	因果自注意力	
Noncausal Self-attention	非因果自注意力	
Multi-head Attention	多头注意力	MHA
Multi-query Attention	多查询注意力	MQA
Grouped-query Attention	分组查询注意力	GQA
Multi-head Latent Attention	多头潜在注意力	MLA
Sparse Attention	稀疏注意力	
Local Attention	局部注意力	
Global Attention	全局注意力	
Dense Attention	稠密注意力	
Exact Attention	精确注意力	
Efficient Attention	高效注意力	
Linear Attention	线性注意力	
Softmax	Softmax 归一化	Softmax
Row-wise Softmax	逐行 Softmax	
Column-wise Softmax	逐列 Softmax	
Online/Blocked Softmax	在线／分块 Softmax	

英文原词	中文译名	缩写／符号
Attention Head	注意力头	
Query Head	查询头	
Key/Value Head	键值头	KV Head
Attention Weight	注意力权重	
Score Matrix	分数矩阵	
Compatibility Score	兼容性分数	
Weighted Aggregation	加权聚合	
Content-dependent Aggregation	依赖内容的聚合	
Weighted Average	加权平均	
Weighted Sum	加权和	
Value Space	值空间	
Representation	表示	
Representation Space	表示空间	
Hidden State	隐藏状态	
Hidden Representation	隐藏表示	
Learned Projection	学习得到的投影	
Linear Projection	线性投影	
Output Projection	输出投影	
Up-projection	升维投影	
Projection Absorption	投影吸收	
Low-rank Joint Compression	低秩联合压缩	
Compressed Latent	压缩潜在表示	
Vector	向量	
Matrix	矩阵	
Matrix Multiplication	矩阵乘法	
Dot Product	点积	
Tensor	张量	
Tensor Shape	张量形状	
Batch Size	批大小	B
Query Length	查询长度	L
Source Length	源长度	S
Per-head Dimension	每个头的维度	
Mask	掩码	

英文原词	中文译名	缩写／符号
Additive Mask	加性掩码	
Boolean Mask	布尔掩码	
Causal Mask	因果掩码	
Padding Mask	填充掩码	
Causal Bias	因果偏置	
Causal Alignment	因果对齐	
Lower Triangular	下三角	
Connectivity Pattern	连接模式	
Local Window	局部窗口	
Logit	逻辑值（归一化前的分数）	
Scaling	缩放	
Normalization	归一化	
Renormalization	重新归一化	
Gradient	梯度	
Mean	均值	
Variance	方差	
Saturated Regime	饱和区	
Sequence-to-sequence	序列到序列	Seq2seq
Neural Machine Translation	神经机器翻译	NMT
Encoder	编码器	
Decoder	解码器	
Context Vector	上下文向量	
Token	词元	
Token Embedding	词元嵌入	
Positional Encoding	位置编码	
Rotary Position Embedding	旋转位置嵌入	RoPE
Decoupled RoPE	解耦 RoPE	
Permutation Equivariance	置换等变性	
Relative Displacement	相对位移	
Absolute Position	绝对位置	
Context Extrapolation	上下文外推	
Autoregressive Language Model	自回归语言模型	
Autoregressive Objective	自回归目标	

英文原词	中文译名	缩写／符号
Decoder-only LLM	仅解码器大语言模型	
Training	训练	
Inference	推理	
Prefill	预填充	
Incremental Decoding	增量解码	
KV Cache	键值缓存	KV Cache
Memory Bandwidth	内存带宽	
Arithmetic Work	算术工作量	
Materialized Intermediate Memory	物化中间量内存	
Memory Traffic	内存流量	
Memory Footprint	内存占用	
Wall-clock Time	实际耗时	
High-bandwidth Memory	高带宽内存	HBM
Static Random-access Memory	静态随机存取存储器	SRAM
Memory Hierarchy	内存层次结构	
IO-aware Exact Attention	感知输入／输出的精确注意力	
Tiling	分块	
Kernel	内核	
Work Partitioning	工作划分	
Thread Block	线程块	
Warp	线程束	
Tensor-core Throughput	张量核心吞吐量	
Pipeline Structure	流水线结构	
Algorithm/Kernel Co-design	算法与内核协同设计	
Dropout	随机失活	
Residual Connection	残差连接	
Feed-forward Network	前馈网络	FFN
Position-wise Feed-forward Layer	逐位置前馈层	
Allocator Overhead	分配器开销	
Activation	激活	
Interpretability	可解释性	
Faithful Explanation	忠实解释	
Faithful Causal Explanation	忠实因果解释	

英文原词	中文译名	缩写／符号
Gradient-based Importance Measure	基于梯度的重要性度量	
Baseline	基线	
Benchmark	基准测试	
FlashAttention	FlashAttention（方法名保留）	
PagedAttention	PagedAttention（方法名保留）	
Longformer	Longformer（模型名保留）	
BigBird	BigBird（模型名保留）	

理解注意力机制

对应原文第 1 页

从查询（Query，Q）、键（Key，K）、值（Value，V）到现代 Transformer 注意力机制（Attention）

本技术手册解释注意力机制计算的内容、其公式为何采用当前形式，以及现代大语言模型（Large Language Model，LLM）系统如何执行该计算。

作者：@techNmak

中文版：专业术语对照表在前，随后按原文页序提供全文译文。正文术语括注原词；公式保留原文矢量内容；图中英文标签另附中文对照。章节编号、引用编号和原文页码均保留。

本手册的学习目标

对应原文第 2 页

读完本手册后，你应当能够看到下式，并解释其中每个对象和运算：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

查询（Query，Q）、键（Key，K）和值（Value，V）来自哪里；QK^T 的每个元素表示什么；为何缩放分数；掩码（Mask）如何改变允许的连接；为何逐行应用 Softmax；以及为何乘以 V 能为每个查询位置产生新的表示（Representation）。

你还应能够解释同一个核心机制如何出现在自注意力（Self-attention）、交叉注意力（Cross-attention）、因果语言模型（Causal Language Model）、多头注意力（Multi-head Attention）、使用键值缓存的解码（KV-cached Decoding）、MQA、GQA、MLA、FlashAttention 和稀疏／局部注意力（Sparse/Local Attention）中。

前置知识（Prerequisites）

你需要基本熟悉向量（Vector）、矩阵（Matrix）、矩阵乘法（Matrix Multiplication）、点积（Dot Product）、Softmax，以及神经网络线性层（Linear Layer）。无需事先了解 Transformer 内部结构。

范围（Scope）

本手册讨论注意力机制（Attention）。涉及 RoPE、键值缓存（KV Caching）、MQA/GQA/MLA 和 FlashAttention，仅为解释它们如何修改、支持或执行注意力计算。完整的推理服务（Inference Serving）、PagedAttention、长上下文训练（Long-context Training）和模型并行（Model Parallelism）值得另写手册。

准确性原则（Accuracy Rule）

具名方法以其原始论文为依据；基础解释与当前斯坦福课程材料交叉核对；实现行为与当前 PyTorch 文档核对。对历史论文的陈述限定在论文实际确立的结论内。[1, 2, 3, 15, 16, 17]

目录

对应原文第 3 页

点击条目即可跳转；以下页码为中文版 PDF 页码。

专业术语翻译对照表 · 第 1 页
封面与译本说明 · 第 6 页
本手册的学习目标 · 第 7 页
1 Transformer 之前：注意力为何出现 · 第 9 页
2 注意力：依赖内容的加权聚合 · 第 10 页
3 记号与张量形状 · 第 11 页
4 Q、K、V 从哪里来 · 第 12 页
5 分数矩阵：QK^T · 第 13 页
6 为什么除以 $\sqrt{d_k}$ · 第 14 页
7 掩码：哪些连接被允许 · 第 15 页
8 Softmax 与值聚合 · 第 16 页
9 手算示例 I：Q、K、V 与原始分数 · 第 17 页
10 手算示例 II：Softmax 权重 · 第 18 页
11 手算示例 III：输出向量 · 第 19 页
12 自注意力与交叉注意力 · 第 20 页
13 多头注意力 · 第 21 页
14 位置：注意力需要顺序信号 · 第 22 页
15 自回归语言模型中的因果注意力 · 第 23 页
16 训练与解码 · 第 24 页
17 键值缓存 · 第 25 页
18 MHA、MQA 与 GQA · 第 26 页
19 MLA：压缩键值表示 · 第 27 页
20 到底什么是二次的 · 第 28 页
21 FlashAttention：相同注意力，不同执行方式 · 第 29 页
22 FlashAttention 的代际演进：优化实际瓶颈 · 第 30 页
23 稀疏与局部注意力 · 第 31 页
24 一个清晰展示过程的 PyTorch 实现 · 第 32 页
25 PyTorch 中用于生产环境的 SDPA · 第 33 页
26 常见实现错误 · 第 34 页
27 常见误解 · 第 35 页
28 注意力权重能作为解释吗 · 第 36 页
29 完整的理解框架 · 第 37 页
参考文献与延伸阅读 · 第 38 页

1 Transformer 之前：注意力为何出现

对应原文第 4 页

早期神经序列到序列（Sequence-to-sequence）系统通常把整个源序列（Source Sequence）编码成一个固定长度向量（Fixed-length Vector），再让解码器（Decoder）根据该表示生成目标序列（Target Sequence）。Bahdanau、Cho 和 Bengio 指出，这种固定向量接口可能成为瓶颈（Bottleneck），尤其当源序列越来越长时。他们的模型让解码器在每个输出步骤形成不同的上下文向量（Context Vector）：按照编码器状态（Encoder State）与当前解码状态的相关程度，对这些状态进行软加权。[1]

当时的关键想法还不是“自注意力（Self-attention）”。解码器使用学习得到的兼容性机制（Compatibility Mechanism），判断哪些源表示应当对当前输出作出更大贡献。随后 Luong、Pham 和 Manning 研究了多种神经机器翻译（Neural Machine Translation，NMT）注意力变体，包括覆盖全部源位置的全局注意力（Global Attention），以及覆盖部分位置的局部注意力（Local Attention）。[2]

2017 年的 Transformer 更彻底地改变了注意力的角色。它不再以循环（Recurrence）或卷积（Convolution）作为序列内信息传递的主要机制，而是围绕注意力与逐位置前馈层（Position-wise Feed-forward Layer）构建编码器和解码器。论文将自注意力描述为：关联同一序列中的不同位置，以计算新的表示。[3]

理解框架（Mental Model）：注意力并非起源于“Transformer 公式”。其经久不变的思想是学习得到的、依赖内容的聚合（Content-dependent Aggregation）：查询决定不同键值对（Key-value Pair）对输出贡献的强弱。Transformer 让一种特定注意力形式成为核心的序列混合运算（Sequence-mixing Operation）。

我们将研究什么

后续内容聚焦缩放点积注意力（Scaled Dot-product Attention），因为 Transformer 和现代大语言模型的注意力内核（Attention Kernel）采用这一形式。更早的加性注意力（Additive Attention）机制在历史上很重要，但其评分函数（Scoring Function）不同。

2 注意力：依赖内容的加权聚合

对应原文第 5 页

注意力函数（Attention Function）可描述为：将一个查询（Query）和一组键值对（Key-value Pair）映射到输出。每个键用于判断其对应值与查询的兼容程度；输出是各个值的加权组合（Weighted Combination）。Transformer 论文直接给出了这一一般性描述。[3]

对一个查询向量 q 和键值对 (k_j, v_j)，缩放点积注意力（Scaled Dot-product Attention）可写为：

$$s_j = \frac{q^\top k_j}{\sqrt{d_k}}, \qquad a_j = \frac{e^{s_j}}{\sum_r e^{s_r}}, \qquad o = \sum_j a_j v_j.$$

分数 s_j 是学习得到的兼容性信号（Compatibility Signal），因为 q 和 k_j 本身来自学习得到的投影（Learned Projection）。Softmax 将分数转为非负权重（Nonnegative Weight），并使允许参与的键的权重之和为 1。输出 o 因而是值空间（Value Space）中的加权平均（Weighted Average）。

这一说法比“查询提出问题，键包含答案”等常见拟人化比喻更精确。比喻或许好记，但并非机制本身。查询和键是学习得到的表示空间（Representation Space）中的向量，其点积提供一个标量兼容性分数（Scalar Compatibility Score）。

理解框架（Mental Model）：键决定权重（Weight），值提供被混合的内容（Content）。这种分工是注意力同时使用 K 和 V 的概念原因。

一个有用的抽象

同时处理多个查询时，注意力变成矩阵计算：

$$S = QK^\top, \qquad A = \mathrm{softmax}(S/\sqrt{d_k}), \qquad O = AV.$$

接下来逐一解释各个对象及其形状（Shape）。

3 记号与张量形状

对应原文第 6 页

设 B 为批大小（Batch Size）；L 为查询位置数（Number of Query Positions）；S 为键／值位置数（Number of Key/Value Positions）；Hq 为查询头数（Number of Query Heads）；Hkv 为键值头数（Number of Key/Value Heads）；dh 为每个头的查询／键维度（Per-head Query/Key Dimension）；dv 为每个头的值维度（Per-head Value Dimension）。

对于单个头（Head）和单个样本，可以忽略批次和头维度，写成：

$$Q \in \mathbb{R}^{L \times d_h}, \quad K \in \mathbb{R}^{S \times d_h}, \quad V \in \mathbb{R}^{S \times d_v}.$$

因此：

$$QK^\top \in \mathbb{R}^{L \times S}, \qquad A \in \mathbb{R}^{L \times S}, \qquad AV \in \mathbb{R}^{L \times d_v}.$$

有两个重要细节。

第一，注意力矩阵（Attention Matrix）并不总是方阵（Square Matrix）。对单个完整序列进行自注意力时，L = S；在交叉注意力中，或者缓存解码（Cached Decoding）中较短查询块关注较长前缀（Prefix）时，L 与 S 可以不同。当前 PyTorch SDPA 文档明确区分查询长度 L 与源长度 S。[17]

第二，现代分组注意力（Grouped Attention）中，Hq 可以不等于 Hkv。标准 MHA 通常具有相同的查询头数和键值头数；MQA 使用一个键值头；GQA 使用介于两者之间的数量。[4, 9]

重要（Important）：不要默认把 n × n 当作注意力权重（Attention Weight）的通用形状。它适用于等长自注意力，却会掩盖交叉注意力和增量推理（Incremental Inference）的情况。

4 Q、K、V 从哪里来

对应原文第 7 页

设注意力层（Attention Layer）的输入是词元表示（Token Representation）组成的矩阵：

$$X \in \mathbb{R}^{n \times d_{\text{model}}}.$$

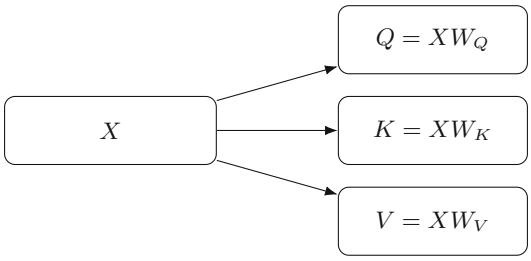
标准自注意力层使用学习得到的线性投影（Linear Projection）：

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V.$$

对一个头，有：

$$W_Q, W_K \in \mathbb{R}^{d_{\text{model}} \times d_h}, \qquad W_V \in \mathbb{R}^{d_{\text{model}} \times d_v}.$$

在多头注意力（Multi-head Attention）中，从概念上说，每个头都有不同的学习投影；实现时也可以先计算一个更大的投影，再重塑（Reshape）为多个头。Transformer 论文用每个头独立的学习投影来定义多头注意力。[3]



Q、K、V 因而不是词元固定携带的语义角色（Semantic Role），而是当前隐藏表示（Hidden Representation）的学习视图（Learned View）。同一词元在不同层和不同头中，可以有不同的查询／键／值向量。

理解框架（Mental Model）：模型学习两个用于判断兼容性的空间（Q 和 K），以及一个用于存放待聚合信息的空间（V）。

5 分数矩阵：QK^T

对应原文第 8 页

取查询行 q_i 与键行 k_j，它们的点积（Dot Product）为：

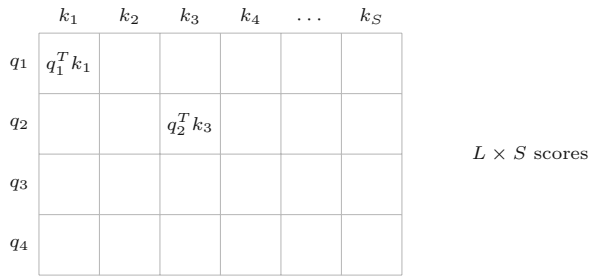
$$S_{ij} = q_i^\top k_j.$$

完整的矩阵乘法（Matrix Multiplication）QK^T 并行计算全部查询与键的点积：

$$QK^\top = \begin{bmatrix} q_1^\top k_1 & q_1^\top k_2 & \cdots & q_1^\top k_S \\ q_2^\top k_1 & q_2^\top k_2 & \cdots & q_2^\top k_S \\ \vdots & \vdots & \ddots & \vdots \\ q_L^\top k_1 & q_L^\top k_2 & \cdots & q_L^\top k_S \end{bmatrix}.$$

每一行对应一个查询位置，每一列对应一个键位置。

点积不是概率（Probability），而是未归一化的兼容性分数（Unnormalized Compatibility Score）。它可以为正或为负，其大小取决于向量的尺度（Scale）和维度（Dimension）。



图中标签：q1⋯q4 为查询；k1⋯kS 为键；“L × S scores”表示 L × S 个分数。标出的格子分别是 $q1^Tk1$ 与 $q2^Tk3$。

重要（Important）：把分数称为“相似度（Similarity）”很方便，但不完整。点积注意力（Dot-product Attention）专门学习投影，使得到的兼容性对任务有用。它不必表现得像通用语义相似度度量（Semantic-similarity Metric）。

6 为什么除以 $\sqrt{d_k}$

对应原文第 9 页

Transformer 在 Softmax 之前缩放查询与键的点积：

$$\widetilde{S} = \frac{QK^\top}{\sqrt{d_k}}.$$

论文给出了具体动机。假设 q 与 k 的各分量是均值（Mean）为 0、方差（Variance）为 1 的独立随机变量（Independent Random Variable），则：

$$q^\top k = \sum_{r=1}^{d_k} q_r k_r$$

其均值为 0，方差为 d_k。随着 d_k 增大，未缩放的点积可能变大。绝对值很大的逻辑值（Logit）会把 Softmax 推入梯度（Gradient）很小的饱和区（Saturated Regime）。除以 $\sqrt{d_k}$，会把这一示例中的方差从 d_k 归一化到约 1。[3]

在上述假设下，简短推导为：

$$\mathrm{Var}(q^\top k) = \sum_{r=1}^{d_k} \mathrm{Var}(q_r k_r) = d_k,$$

因此：

$$\mathrm{Var}\left(\frac{q^\top k}{\sqrt{d_k}}\right) \approx 1.$$

重要（Important）：这一推导解释的是动机，并不是说训练后的 Transformer 中，学习得到的查询／键分量始终是独立、零均值、单位方差的随机变量。论文明确将这些假设用于示例，说明点积为何会随维度增大。

理解框架（Mental Model）：缩放（Scaling）让分数大小随着维度增长仍处于 Softmax 能有效工作的范围；它并非将 Q 或 K 归一化为单位向量（Unit Vector）。

7 掩码：哪些连接被允许

对应原文第 10 页

Softmax 之前，注意力实现可以向分数矩阵添加掩码（Mask）或偏置（Bias）。对于因果语言建模（Causal Language Modeling），未来键位置不得影响较早的查询位置。常用加性掩码（Additive Mask）为：

$$M_{ij} = \begin{cases} 0, & j \le i, \\ -\infty, & j > i. \end{cases}$$

于是：

$$A = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right).$$

由于 exp(-∞) = 0，被掩蔽项在 Softmax 后得到零概率。

对于长度为 4 的方形因果注意力矩阵，允许连接的模式为：

$$\begin{bmatrix} \checkmark & \times & \times & \times \\ \checkmark & \checkmark & \times & \times \\ \checkmark & \checkmark & \checkmark & \times \\ \checkmark & \checkmark & \checkmark & \checkmark \end{bmatrix}.$$

原始 Transformer 解码器将 Softmax 之前的非法连接设为 -∞，从而实现自回归掩蔽（Autoregressive Masking）。[3]

掩码的用途不限于因果约束。填充掩码（Padding Mask）可以阻止关注填充词元（Padding Token）；结构化注意力（Structured Attention）可以把连接限制在局部窗口（Local Window）、块（Block）或其他模式。当前 PyTorch SDPA 接受布尔掩码（Boolean Mask）或加性浮点掩码（Additive Floating-point Mask），并单独支持因果模式（Causal Mode）。[17]

重要（Important）：在非方形注意力（Non-square Attention，L ≠ S）中，“下三角（Lower Triangular）”需要指定对齐约定（Alignment Convention）。当前 PyTorch 文档明确区分方形与非方形因果偏置的行为。未核对 API 预期前，不要盲目构建 L × S 三角掩码。

8 Softmax 与值聚合

对应原文第 11 页

完成缩放和掩蔽后，对每个查询行沿键维度应用 Softmax：

$$A_{ij} = \frac{\exp(\widetilde{S}_{ij})}{\sum_{r=1}^{S} \exp(\widetilde{S}_{ir})}.$$

因此，对每个查询 i：

$$A_{ij} \geq 0, \qquad \sum_{j=1}^{S} A_{ij} = 1.$$

查询位置 i 的输出为：

$$o_i = \sum_{j=1}^{S} A_{ij} v_j.$$

写成矩阵形式：

$$O = AV.$$

最后这次乘法容易被忽略，但它才是注意力真正传递信息的地方。分数矩阵只决定每个值贡献多少；值向量（Value Vector）才是被混合进新表示的内容。

如果某个查询赋予三个值的权重为 (0.7, 0.2, 0.1)，它的输出为：

$$0.7v_1 + 0.2v_2 + 0.1v_3.$$

输出仍位于值向量空间（Value-vector Space），而不是分数空间（Score Space）。

理解框架（Mental Model）：分数回答“多少”；值回答“聚合什么”。完整的注意力运算是依赖内容的混合算子（Content-dependent Mixing Operator）。

为什么逐行处理

每个查询独立地产生一个覆盖其允许键位置的分布（Distribution）。逐列 Softmax（Column-wise Softmax）会改为在查询之间归一化，描述的是另一种运算。

9 手算示例 I：Q、K、V 与原始分数

对应原文第 12 页

现在完整手算一个因果注意力头（Causal Attention Head）。取：

$$Q = K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}, \qquad V = \begin{bmatrix} 1 & 0 \\ 0 & 2 \\ 3 & 1 \end{bmatrix}, \qquad d_k = 2.$$

共有三个查询位置和三个键／值位置。查询与键的点积为：

$$QK^\top = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 2 \end{bmatrix}.$$

除以 $\sqrt{2}$ 后：

$$\frac{QK^\top}{\sqrt{2}} \approx \begin{bmatrix} 0.7071 & 0 & 0.7071 \\ 0 & 0.7071 & 0.7071 \\ 0.7071 & 0.7071 & 1.4142 \end{bmatrix}.$$

现在加入因果掩码（Causal Mask）。查询 1 只能使用键 1；查询 2 可以使用键 1 和键 2；查询 3 可以使用全部三个键：

$$\widetilde{S}_{\text{causal}} \approx \begin{bmatrix} 0.7071 & -\infty & -\infty \\ 0 & 0.7071 & -\infty \\ 0.7071 & 0.7071 & 1.4142 \end{bmatrix}.$$

本手算示例的所有数值，均在原稿排版前通过程序独立重算。

10 手算示例 II：Softmax 权重

对应原文第 13 页

逐行应用 Softmax。

对第一个查询，只有一个位置合法，因此：

$$A_1 = [1, 0, 0].$$

对第二个查询，合法逻辑值（Logit）为 $[0, 0.7071]$：

$$A_2 \approx [0.3302,\ 0.6698,\ 0].$$

对第三个查询，逻辑值为 $[0.7071, 0.7071, 1.4142]$：

$$A_3 \approx [0.2483,\ 0.2483,\ 0.5035].$$

因此：

$$A \approx \begin{bmatrix} 1 & 0 & 0 \\ 0.3302 & 0.6698 & 0 \\ 0.2483 & 0.2483 & 0.5035 \end{bmatrix}.$$

每行之和为 1（允许舍入误差），所有因果约束禁止的位置权重均为 0。

第三行表示：第三个查询把约一半的注意力权重（Attention Weight）分配给第三个值，对此前的两个值各分配约四分之一。这些数字不是我们额外添加的注释，而是这一玩具示例（Toy Setup）中学习兼容性计算经归一化后的结果。

重要（Important）：在训练后的模型中，注意力权重产生于许多头和层内部，并与残差连接（Residual Connection）、归一化（Normalization）、前馈网络（Feed-forward Network）及后续计算相互作用。较高的注意力权重不应自动被视为模型预测（Prediction）的完整解释，后文还会讨论。

11 手算示例 III：输出向量

对应原文第 14 页

最后，将注意力权重乘以值矩阵（Value Matrix）：

$$O = AV.$$

使用：

$$V = \begin{bmatrix} 1 & 0 \\ 0 & 2 \\ 3 & 1 \end{bmatrix},$$

得到：

$$O \approx \begin{bmatrix} 1 & 0 \\ 0.3302 & 1.3395 \\ 1.7587 & 1.0000 \end{bmatrix}.$$

例如第二行为：

$$0.3302[1,0] + 0.6698[0,2] = [0.3302, 1.3396] \text{ (rounding)}.$$

第三行为：

$$0.2483[1,0] + 0.2483[0,2] + 0.5035[3,1] \approx [1.7587, 1.0000].$$

完整的注意力机制现已显现：

$$Q, K \longrightarrow QK^\top \longrightarrow \frac{QK^\top}{\sqrt{d_k}} \longrightarrow \text{mask} \longrightarrow \text{softmax} \longrightarrow A \longrightarrow AV.$$

上图流程中的 mask 指掩码，softmax 指 Softmax 归一化；前式中的 rounding 表示舍入。

理解框架（Mental Model）：如果你能解释这个三词元示例中每个数字的来源，核心注意力公式就不再是黑箱（Black Box）。后续内容改变的只是 Q/K/V 的来源、允许的连接、头结构、位置处理，或高效执行这一计算的方式。

译注：原文输出矩阵给出 1.3395，而用已舍入权重 0.6698 计算的第二行示例给出 1.3396；两者来自不同阶段的舍入。这里保留原文公式，未将这一末位差异改写为计算错误。

12 自注意力与交叉注意力

对应原文第 15 页

注意力公式不要求 Q、K、V 来自同一序列。

自注意力（Self-attention）

在自注意力中，Q、K、V 是同一序列表示（或同一层输入）的投影：

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V.$$

Transformer 编码器采用这种自注意力。因果解码器（Causal Decoder）也使用自注意力，但会掩蔽未来位置。[3]

交叉注意力（Cross-attention）

在编码器－解码器注意力（Encoder-decoder Attention）中，查询来自解码器一侧，键和值来自编码器输出：

$$Q = YW_Q, \qquad K = XW_K, \qquad V = XW_V.$$

如果目标／查询序列长度为 L、源序列长度为 S，分数矩阵形状就是 L × S。

类型	Q 的来源	K、V 的来源
自注意力（Self-attention）	同一序列	同一序列
交叉注意力（Cross-attention）	查询／目标流	源／记忆流
因果自注意力（Causal Self-attention）	同一序列	同一序列，附掩码

重要（Important）：“自注意力”描述 Q/K/V 来自哪里；“因果注意力（Causal Attention）”描述连接约束。两者不是同义词：自注意力可以是双向（Bidirectional）的，也可以是因果的。

13 多头注意力

对应原文第 16 页

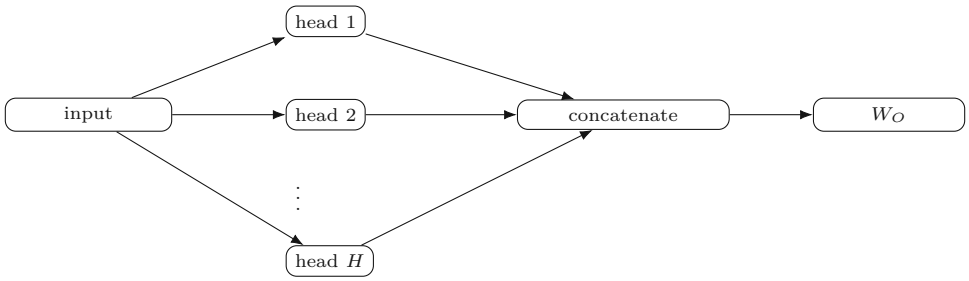
单个注意力头（Attention Head）产生一种学习得到的兼容性与聚合模式。多头注意力（Multi-head Attention）使用不同的学习投影，并行执行多次注意力运算，然后拼接（Concatenate）各头输出，再应用输出投影（Output Projection）。Transformer 定义：

$$\mathrm{MHA}(Q,K,V) = \mathrm{Concat}(\mathrm{head}_1,\ldots,\mathrm{head}_H)W_O,$$

其中：

$$\mathrm{head}_h = \mathrm{Attention}(QW_h^Q, KW_h^K, VW_h^V).$$

[3]



图中标签：input 为输入；head 1⋯head H 为第 1⋯H 个头；concatenate 为拼接；W_O 为输出投影矩阵。

如果保持模型维度 dmodel 固定，每个头的维度约为 dmodel/H，那么增加头数并不会简单地把全维度注意力成本乘以 H，因为每个头更窄。原始 Transformer 用这一设计，使总成本与单个全维度头相近。[3]

重要（Important）：不要假设每个头都对应一个清晰、可供人类解释的概念。不同头可以学习不同模式，但头的语义（Head Semantics）是涌现（Emergent）的，不能保证对应简单标签。

14 位置：注意力需要顺序信号

对应原文第 17 页

不包含任何位置相关信号（Position-dependent Signal）的普通非因果自注意力（Noncausal Self-attention）具有置换等变性（Permutation Equivariance）：如果输入行发生置换，输出也按相同方式置换。它本身不知道某个词元是“第三个”，也不知道另一个词元“相距五个位置”。因此，原始 Transformer 向词元嵌入（Token Embedding）添加位置编码（Positional Encoding）。[3]

现代仅解码器大语言模型（Decoder-only LLM）常用旋转位置嵌入（Rotary Position Embedding，RoPE）。RoPE 按与位置有关的角度旋转查询与键的分量。在一个简化二维块中：

$$R_m(\theta) = \begin{bmatrix} \cos(m\theta) & -\sin(m\theta) \\ \sin(m\theta) & \cos(m\theta) \end{bmatrix}.$$

若 q_m = R_m q 且 k_n = R_n k，则：

$$q_m^\top k_n = q^\top R_m^\top R_n k = q^\top R_{n-m} k,$$

因此，注意力交互（Attention Interaction）包含相对位移（Relative Displacement）信息。RoFormer 将 RoPE 描述为通过旋转编码绝对位置（Absolute Position），同时在自注意力中引入明确的相对位置依赖（Relative-position Dependence）。[7]

两个细节很重要。

第一，位置编码不是注意力本身。它改变表示或 Q/K 交互，使注意力可以利用位置。

第二，因果掩码本身会引入有方向的顺序约束，但它本身并不能提供丰富的距离或绝对位置表示。

重要（Important）：数学上能够在超出训练长度的位置计算 RoPE，并不保证可靠的长上下文行为（Long-context Behavior）。上下文外推（Context Extrapolation）依赖训练和位置配置，不能只靠 RoPE。

15 自回归语言模型中的因果注意力

对应原文第 18 页

自回归语言模型（Autoregressive Language Model）将词元序列分解为：

$$p(x_{1:T}) = \prod_{t=1}^{T} p(x_t \mid x_{<t}).$$

在完整序列上训练仅解码器 Transformer 时，模型可以并行处理许多位置，因为因果掩码阻止位置 t 的表示使用未来词元。每一层的掩蔽自注意力（Masked Self-attention）都强制执行允许的依赖模式（Dependency Pattern）。[3]

对查询位置 t，注意力可以使用位置 1,⋯,t 的键／值，但不能使用位置 t+1,⋯,T。

这是训练（Training）与生成（Generation）在计算上差异很大的原因之一。训练时，完整的已知序列可用，大型矩阵乘法可以一起处理所有查询位置。生成时，只有生成词元 t 后，词元 t+1 才会存在，因此解码（Decoding）对生成词元而言是顺序进行的。

理解框架（Mental Model）：因果掩蔽（Causal Masking）使跨位置的并行训练兼容自回归目标（Autoregressive Objective）。它并不能让自回归生成变成并行，因为未来生成词元还未知。

注意力计算本身仍是缩放点积注意力。推理（Inference）时改变的是：此前的计算有多少可以复用。

16 训练与解码

对应原文第 19 页

考虑一个处理长度为 T 的前缀的因果解码器层（Causal Decoder Layer）。

训练／完整前缀处理（Training / Full-prefix Processing）

对完整训练序列或提示词预填充（Prompt Prefill），许多查询位置会一起处理。稠密注意力（Dense Attention）形成相关查询与键之间的交互，通常通过大型批量矩阵运算（Batched Matrix Operation）完成。

增量解码（Incremental Decoding）

处理完前缀后，模型生成一个新词元。其新隐藏状态（Hidden State）在每层产生新的查询、键和值。关键在于：较早位置的键和值无需从头重新计算。由于因果注意力，较早词元的表示已经在无法访问未来词元的情况下确定。

这一观察引出了键值缓存（KV Cache）。

项目	预填充／类似训练的处理	增量解码
处理的查询	许多位置	通常一／少量新位置
过去的 K/V	在本次处理中计算	从缓存复用
序列间并行性（Parallelism）	高	生成仍按顺序进行
典型瓶颈侧重	计算和内存	缓存带宽／容量与每词元延迟

重要（Important）：“预填充（Prefill）”和“解码（Decode）”描述的是服务阶段（Serving Phase），并非不同的注意力数学定义。它们使同一模型呈现不同形状和硬件瓶颈（Hardware Bottleneck）。

17 键值缓存

对应原文第 20 页

在因果增量解码中，每层保存此前词元产生的键和值向量。新词元到来时，该层计算新词元的查询／键／值，将其 K/V 追加到缓存（Cache），再用新查询关注缓存中的键和值。

对标准注意力，每个词元的缓存元素数量近似为：

$$2 \times L_{\text{layers}} \times H_{kv} \times d_h,$$

其中因子 2 对应 K 和 V。要得到字节数（Bytes），再乘以每个元素的存储字节数。

例如，32 层、dh = 128、K/V 每元素占 2 字节时：

键值头数（KV Heads）	每词元缓存	8192 个词元的缓存
32	512 KiB	4.0 GiB
8	128 KiB	1.0 GiB
1	16 KiB	0.125 GiB

此例只计算 K/V 存储，不包括分配器开销（Allocator Overhead）、元数据（Metadata）、其他激活（Activation）、权重（Weight）以及实现特有的表示。

键值缓存（KV Caching）避免了每生成一个词元就重新计算整个前缀的 K/V。但它不能使稠密注意力独立于上下文长度（Context Length）：新查询仍需与缓存前缀交互，缓存内存也随序列长度增长。

重要（Important）：对标准稠密注意力而言，“KV 缓存让解码变成 O(1)”是错误的。尽管过去的 K/V 投影被复用，新词元的注意力工作量仍随缓存键／值位置数增长。

18 MHA、MQA 与 GQA

对应原文第 21 页

键值缓存让键值头数成为推理系统（Inference System）的决策，而不仅仅是架构细节（Architectural Detail）。

多头注意力（Multi-head Attention，MHA）

标准 MHA 的每个注意力头都有独立 K/V 投影。如果有 H 个查询头，通常也有 H 个键值头。

多查询注意力（Multi-query Attention，MQA）

Shazeer 提出 MQA，以加快 Transformer 的增量解码。它保留多个查询头，却让这些查询头共享一个键头和一个值头，从而大幅缩小缓存的 K/V 张量（Tensor），并减少内存带宽（Memory Bandwidth）需求。[4]

分组查询注意力（Grouped-query Attention，GQA）

GQA 通过采用中间数量的键值头来推广 MQA：键值头多于一个，却少于查询头。查询头被划分为多个组，每组共享一个键值头。Ainslie 等人提出 GQA，目的是在其试验中接近 MQA 的推理效率，同时保留更接近 MHA 的质量。[9]

MHA：多个查询头（Query Heads）分别对应多个键值头（KV Heads）。；GQA：多个查询头按组共享键值组（KV Groups）。；MQA：所有查询头共享一个键值头（One Shared KV Head）。

理解框架（Mental Model）：MQA/GQA 减少的是键值头，而不是查询头。查询头的数量仍可较多，同时共享缓存的键值表示（Key/Value Representation）。

19 MLA：压缩键值表示

对应原文第 22 页

DeepSeek-V2 引入的多头潜在注意力（Multi-head Latent Attention，MLA），采用与 MQA/GQA 不同的方式减少键值缓存。MLA 并非主要共享 K/V 头，而是对键和值信息进行低秩联合压缩（Low-rank Joint Compression）。[11]

按论文记号，压缩潜在表示（Compressed Latent）为：

$$c_t^{KV} = W^{DKV} h_t,$$

随后，可通过学习得到的升维投影（Up-projection）推导键／值内容。核心动机是：缓存状态可显著小于传统的逐头键和值存储。

不过，准确说明必须包含一个重要限定：DeepSeek-V2 还使用解耦 RoPE（Decoupled RoPE）。与 RoPE 相关的位置敏感键分量（Position-sensitive Key Component）被单独处理，因为在压缩键路径内部直接应用 RoPE，会阻止高效推理中使用的某些投影吸收（Projection Absorption）。因此，论文中的缓存同时包含压缩潜在表示和解耦的位置键分量；完整实现并非字面意义上只存一个裸潜在向量。[11]

机制	键值缓存的主要思想	键值头
MHA	保存每个头的 K 和 V	多个
MQA	共享一个 K/V 头	一个
GQA	组内共享 K/V	中间数量
MLA	低秩联合压缩 KV，另加位置分量	不同的形式

重要（Important）：MLA 并非只是“头更少的 GQA”。它改变了缓存中的表示，以及推理时采用的投影代数（Projection Algebra）。

20 到底什么是二次的

对应原文第 23 页

“注意力是二次的（Quadratic）”这一说法，压缩了几个不同的资源问题。

对长度为 n 的稠密自注意力，在表示／头维度固定时，原始 Transformer 分析给出的每层自注意力项为 $O(n^2d)$。[3]

但必须区分三个量：

1. 算术工作量（Arithmetic Work）：计算全部稠密查询与键交互，再将其作用于值，会产生关于序列长度的二次项。

2. 物化中间量内存（Materialized Intermediate Memory）：朴素实现（Naive Implementation）可能显式保存 n × n 的分数／概率矩阵，因此中间存储呈二次增长。

3. 内存流量（Memory Traffic）：即便算术量不变，在 GPU 的不同内存层级（Memory Level）间读写中间量，仍可能主导运行时间（Runtime）。

这些区别解释了为什么下面两句话可以同时成立：

- 稠密注意力仍执行二次数量的成对交互（Pairwise Interaction）。

- 精确注意力（Exact Attention）实现可以避免在高带宽内存（High-bandwidth Memory）中物化整个二次大小的注意力矩阵，并运行得快得多。

第二句话正是 FlashAttention 的核心。

理解框架（Mental Model）：复杂度（Complexity）、内存占用（Memory Footprint）、内存流量（Memory Traffic）和实际耗时（Wall-clock Time）彼此相关，却不可互换。不要把其中一个当作另一个的同义词。

21 FlashAttention：相同注意力，不同执行方式

对应原文第 24 页

FlashAttention 是一种感知输入／输出的精确注意力算法（IO-aware Exact Attention Algorithm）。其关键洞见是明确考虑 GPU 内存层次结构（Memory Hierarchy）：高带宽内存（High-bandwidth Memory，HBM）与更快的片上静态随机存取存储器（On-chip SRAM）之间的传输，可能是主要成本。FlashAttention 不在 HBM 中物化完整注意力矩阵，而是将计算分块（Tiling），让 Q/K/V 数据块和中间统计量（Intermediate Statistics）在片上处理，减少 HBM 读写。[8]

数学目标仍然是缩放点积注意力。算法使用在线／分块 Softmax（Online/Blocked Softmax）策略，因而无需在 HBM 中一次存储全部分数，就能组合出精确结果。

HBM：Q、K、V、输出（Output）；片上 SRAM／寄存器（Registers）：分块（Tiles）；矩阵乘法（Matrix Multiply）＋Softmax 更新（Updates）；更少的大容量内存传输（Fewer Large-memory Transfers）

原始论文强调减少 HBM 访问，而不是用近似（Approximation）替换稠密注意力。斯坦福 CS336 将 FlashAttention 用作系统练习，正是为了把 GPU 内存层次结构与注意力性能（Performance）联系起来。[8, 16]

重要（Important）：FlashAttention 不会把标准稠密注意力的算术复杂度从 $O(n^2)$ 变成 $O(n)$。它在计算精确稠密注意力的同时，减少内存流量，并避免在 HBM 中物化二次大小的注意力矩阵。

22 FlashAttention 的代际演进：优化实际瓶颈

对应原文第 25 页

经久不变的概念是感知输入／输出的精确注意力（IO-aware Exact Attention）。后续 FlashAttention 版本改进的是如何把这一计算映射到更新的 GPU 硬件。

FlashAttention-2 改进了并行性（Parallelism）和工作划分（Work Partitioning），包括在线程块（Thread Block）和线程束（Warp）之间更好地分配工作，以及减少非矩阵乘法运算（Non-matrix-multiply Operation）。[10]

2026 年面向 Blackwell 级 GPU 推出的 FlashAttention-4，针对的是不同的硬件平衡：张量核心吞吐量（Tensor-core Throughput）的增长快于部分周边资源，瓶颈因而转移到 Softmax／非矩阵乘法工作、共享内存流量（Shared-memory Traffic）和流水线结构（Pipeline Structure）。它的贡献并非新的注意力定义，而是针对变化后的机器，进行考虑硬件的算法与内核协同设计（Algorithm/Kernel Co-design）。[12]

这是一个重要的工程经验：

> 理解框架（Mental Model）：算法的数学形式可以不变，但其最快实现会随硬件改变。“注意力”是数学运算；“FlashAttention-N”则是一族越来越注重硬件特点的执行方法。

要建立持久有效的理解框架，请记住：

1. 避免不必要的物化（Materialization）。

2. 对数据分块，以利用更快的内存。

3. 融合（Fuse）相容的运算。

4. 持续向昂贵的计算单元供给工作。

5. 当硬件资源比例改变时，重新评估瓶颈。

此处有意省略各代 FlashAttention 的基准测试（Benchmark）数值，因为它们取决于硬件、形状、精度（Precision）和软件版本。

23 稀疏与局部注意力

对应原文第 26 页

稠密注意力将每个查询连接到每个键，对等长自注意力产生 $O(n^2)$ 个成对交互。另一类方法改变的是连接模式（Connectivity Pattern）本身。

Longformer 将局部窗口注意力（Local Windowed Attention）与由任务需求决定的全局注意力（Global Attention）结合。在其固定窗口模式（Fixed-window Pattern）下，注意力成本随序列长度线性增长（Linear Scaling）。[5]

BigBird 将局部（Local）、全局（Global）、随机连接（Random Connection）等稀疏组成部分结合起来，在减少完整二次依赖的同时，仍在其设计条件下保留重要的理论性质（Theoretical Property）。[6]

这些方法在概念上不同于 FlashAttention：

方法	改变什么	是否为精确的完整稠密注意力
FlashAttention	执行／输入输出策略	是
局部／稀疏注意力（Local/Sparse Attention）	计算哪些查询与键的组合	否，连接受限制或结构化

稀疏注意力（Sparse Attention）通过不计算所有组合来减少算术工作量。代价是模型在单层中不再具有不受限制的直接连接；模式设计很重要。

重要（Important）：“高效注意力（Efficient Attention）”不是单一技术。一些方法保留精确稠密注意力，但更好地执行它；另一些方法改变注意力模式（Attention Pattern）或采用近似，以减少交互数量。

24 一个清晰展示过程的 PyTorch 实现

对应原文第 27 页

下面的代码对应简单单头／多头张量布局（Tensor Layout）的数学过程。它有意把步骤写清楚，而非追求优化。

```
import math
import torch

# q, k, v: (..., L/S, d_head)
scores = q @ k.transpose(-2, -1)
scores = scores / math.sqrt(q.size(-1))

if causal:
   L, S = q.size(-2), k.size(-2)
   # 此简单掩码假设为对齐的方形情形。
   mask = torch.triu(
      torch.ones(L, S, dtype=torch.bool, device=q.device),
      diagonal=1,
   )
   scores = scores.masked_fill(mask, float("-inf"))

weights = torch.softmax(scores, dim=-1)
out = weights @ v
```

代码明确展示了概念步骤：

QK^T → 缩放（Scale）→ 掩码（Mask）→ Softmax → AV。

它没有展示生产环境内核（Production Kernel）如何融合运算、处理数值稳定的分块 Softmax（Numerically Stable Blockwise Softmax）、选择硬件后端（Hardware Backend）、在非方形情形下表示因果偏置（Causal Bias），或支持分组查询头（Grouped Query Head）。

重要（Important）：这里的简单三角掩码（Triangular Mask）适用于对齐的方形因果示例。对真实的非方形缓存解码，应使用明确记录因果对齐语义（Causal Alignment Semantics）的库 API，而不是假设这一掩码普遍正确。

25 PyTorch 中用于生产环境的 SDPA

对应原文第 28 页

当前 PyTorch 以下列接口提供缩放点积注意力（Scaled Dot-product Attention，SDPA）：

```
torch.nn.functional.scaled_dot_product_attention(
    query, key, value,
    attn_mask=None,
    dropout_p=0.0,
    is_causal=False,
    scale=None,
    enable_gqa=False,
)
```

文档规定了查询长度 L、源／键长度 S、可选布尔或加性掩码、默认缩放 1/√d、因果偏置行为、随机失活（Dropout），以及实验性的分组查询支持。根据输入和后端（Backend），PyTorch 可以分派（Dispatch）到优化实现，而不必逐字执行前面的教学 Python 步骤。[17]

由此可以得到一些实现经验：

- API 语义（API Semantics）很重要：不同库的掩码不可互换。在当前 PyTorch SDPA 中，布尔 attn_mask 的 True 表示该位置参与注意力；这与 MultiheadAttention 的布尔 key_padding_mask 约定相反。

- 在训练与评估（Evaluation）逻辑之间切换时，必须显式控制 Dropout 行为。

- 当前实现中的 GQA 有形状和整除约束（Divisibility Constraint）。

- 内核选择（Kernel Selection）可能依赖设备（Device）、数据类型（dtype）、形状、掩码和功能支持。

实现 API 会演进。本手册中的数学原理长期有效；生产代码应始终对照已安装框架版本的文档核查。

26 常见实现错误

对应原文第 29 页

1. 在错误维度上应用 Softmax

注意力对每个查询沿键维度归一化。沿查询行应用 Softmax 会改变运算。

2. 忘记缩放

未缩放点积注意力不属于 Transformer 的公式形式，而且可能随头维度变化而表现不同。

3. 在 Softmax 之后掩蔽

在 Softmax 后把概率设为 0，却不重新归一化（Renormalize），通常不等同于在归一化之前排除这些逻辑值（Logit）。

4. 对 L ≠ S 使用方形掩码假设

缓存解码和交叉注意力的查询／键长度常常不同。因果对齐（Causal Alignment）必须符合框架约定。

5. 混淆头数与键值头数

在 GQA 中，查询头和键／值头不同。形状逻辑与缓存公式必须用 Hkv 计算 K/V 存储。

6. 把注意力权重当作层输出

权重是中间量（Intermediate）。输出是 AV；在多头注意力中，随后还要拼接／输出投影，再执行周围 Transformer 块（Transformer Block）的运算。

7. 比较“注意力内存”却不定义统计内容

权重、临时分数矩阵（Temporary Score Matrix）、键值缓存、激活和分配器开销，是不同的量。

理解框架（Mental Model）：多数注意力错误是形状／轴（Axis）／语义错误，而不是记不住主公式。请明确跟踪每个维度。

27 常见误解

对应原文第 30 页

误解（Misconception）	纠正（Correction）
“Q 是问题；K 是答案。”	Q/K 是学习得到的向量投影，其点积提供兼容性。比喻并非机制。
“注意力只是相似度搜索（Similarity Search）。”	点积是学习得到的、任务特定的兼容性分数；随后聚合值。
“注意力权重就是输出。”	它们是系数（Coefficient）；输出是 V 的加权和（Weighted Sum）。
“自注意力就是因果注意力。”	自注意力表示 Q/K/V 来自同一信息流（Stream）；因果性是掩码／连接约束。
“FlashAttention 是线性注意力（Linear Attention）。”	FlashAttention 以感知输入输出的执行策略，计算精确稠密注意力。
“KV 缓存使注意力成为常数时间（Constant Time）。”	它复用过去的 K/V，但新稠密查询仍需与不断增长的前缀交互。
“MQA/GQA 减少查询头。”	其定义性改变是相对于查询头共享／减少键值头。
“RoPE 提供无限上下文（Unlimited Context）。”	位置表示本身不能保证训练得到的长上下文能力。

好的技术解释应尽早消除这些简化说法，因为后续系统概念直接建立在这些区别之上。

28 注意力权重能作为解释吗

对应原文第 31 页

注意力权重（Attention Weight）的可视化往往很有吸引力：它们形成输入位置上的分布，所以很容易把高权重理解成“模型因为这个词元重要而使用了它”。没有额外分析，这一结论通常不能成立。

Jain 和 Wallace 在多种自然语言处理（Natural Language Processing，NLP）模型中检验了注意力权重能否作为解释（Explanation）。他们发现，某些情况下，学习到的注意力与基于梯度的重要性度量（Gradient-based Importance Measure）关联很弱；而且差异很大的注意力分布，也能产生相近预测。他们认为，标准注意力权重不应自动被当作忠实解释（Faithful Explanation）。[13]

Wiegreffe 和 Pinter 对一概否定提出质疑。他们认为，答案取决于解释的定义，以及是否使用适当的基线（Baseline）和测试（Test）。他们提出了一些诊断条件（Diagnostic Condition），说明在何种情况下注意力仍能提供有用信息。[14]

因此，最稳妥的技术结论比任何一句口号都更有限：

> 理解框架（Mental Model）：注意力权重是模型内部量（Internal Model Quantity），有时可以支持分析，但它们并非自动成为模型预测的忠实因果解释（Faithful Causal Explanation）。可解释性（Interpretability）主张需要方法和验证，不能仅靠画出权重。

这一细致区别在现代 Transformer 中尤为重要，因为许多头／层、残差路径（Residual Pathway）、前馈块（Feed-forward Block）和后续计算共同决定最终输出。

29 完整的理解框架

对应原文第 32 页

把注意力记成六个步骤最容易理解。

1. 投影（Project）：将隐藏状态映射为学习得到的查询、键和值表示。

2. 比较（Compare）：将每个查询与允许的键比较，通常通过点积完成。

3. 缩放（Scale）：在缩放点积注意力中，将点积除以 $\sqrt{d_k}$。

4. 约束（Constrain）：掩码或偏置决定允许或偏向哪些交互。

5. 归一化（Normalize）：逐行 Softmax（Row-wise Softmax）将分数转为键上的权重。

6. 聚合（Aggregate）：这些权重将对应值混合为查询的新表示。

本手册中的其他内容，改变的是周边维度之一：

- 多头注意力（Multi-head Attention）在多个学习得到的子空间（Learned Subspace）中重复该运算。

- RoPE／位置方法（Position Method）让注意力感知位置。

- 因果掩蔽（Causal Masking）强制执行自回归信息流（Autoregressive Information Flow）。

- 键值缓存（KV Caching）在增量解码期间复用过去的 K/V。

- MQA/GQA 减少键值头数量，以及缓存／带宽成本。

- MLA 采用不同的投影策略压缩缓存的键值表示。

- FlashAttention 以成本更低的内存流量执行精确稠密注意力。

- 稀疏／局部注意力（Sparse/Local Attention）改变计算哪些组合，以降低稠密连接成本。

如果这些区别已清晰，你就不仅仅知道注意力公式，还拥有理解现代大语言模型架构（LLM Architecture）与推理系统为何不断改变公式周边结构所需的概念基础。

参考文献与延伸阅读

对应原文第 33 页

参考文献（References）

[1] Dzmitry Bahdanau, Kyunghyun Cho, Yoshua Bengio.《通过联合学习对齐与翻译进行神经机器翻译》（Neural Machine Translation by Jointly Learning to Align and Translate）。arXiv:1409.0473；ICLR 2015。

[2] Minh-Thang Luong, Hieu Pham, Christopher D. Manning.《基于注意力的神经机器翻译的有效方法》（Effective Approaches to Attention-based Neural Machine Translation）。EMNLP 2015。arXiv:1508.04025。

[3] Ashish Vaswani 等。《注意力就是你所需要的一切》（Attention Is All You Need）。NeurIPS 2017。arXiv:1706.03762。

[4] Noam Shazeer.《快速 Transformer 解码：只需一个写入头》（Fast Transformer Decoding: One Write-Head is All You Need）。arXiv:1911.02150，2019。

[5] Iz Beltagy, Matthew E. Peters, Arman Cohan.《Longformer：长文档 Transformer》（Longformer: The Long-Document Transformer）。arXiv:2004.05150，2020。

[6] Manzil Zaheer 等。《Big Bird：用于更长序列的 Transformer》（Big Bird: Transformers for Longer Sequences）。NeurIPS 2020。arXiv:2007.14062。

[7] Jianlin Su 等。《RoFormer：采用旋转位置嵌入增强的 Transformer》（RoFormer: Enhanced Transformer with Rotary Position Embedding）。arXiv:2104.09864，2021。

[8] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, Christopher Re.《FlashAttention：感知输入输出的快速、内存高效精确注意力》（FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness）。NeurIPS 2022。arXiv:2205.14135。

[9] Joshua Ainslie 等。《GQA：从多头检查点训练广义多查询 Transformer 模型》（GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints）。EMNLP 2023。arXiv:2305.13245。

[10] Tri Dao.《FlashAttention-2：通过更好的并行性与工作划分加速注意力》（FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning）。arXiv:2307.08691，2023。

[11] DeepSeek-AI 等。《DeepSeek-V2：强大、经济、高效的混合专家语言模型》（DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model）。arXiv:2405.04434，2024。

[12] Ted Zadouri 等。《FlashAttention-4：面向非对称硬件扩展的算法与内核流水线协同设计》（FlashAttention-4: Algorithm and Kernel Pipelining Co-Design for Asymmetric Hardware Scaling）。arXiv:2603.05451，2026。

[13] Sarthak Jain, Byron C. Wallace.《注意力并非解释》（Attention is not Explanation）。NAACL-HLT 2019。

[14] Sarah Wiegreffe, Yuval Pinter.《注意力并非不是解释》（Attention is not not Explanation）。EMNLP-IJCNLP 2019。

[15] Stanford CS224N。《深度学习自然语言处理》（Natural Language Processing with Deep Learning），2026 年冬季课程材料；作业 3：自注意力与 Transformer（Self-Attention and Transformers）。

[16] Stanford CS336。《从零构建语言模型》（Language Modeling from Scratch），当前课程材料；系统作业包括一个 Triton FlashAttention-2 实现。

[17] PyTorch 文档。torch.nn.functional.scaled_dot_product_attention。原稿参考当前文档，核对 API 形状、掩码、因果性、缩放、Dropout 和 GQA 行为。

研究说明（Research Note）：公式、张量形状陈述、手算数值示例及对实现敏感的陈述，均在原稿准备期间独立核查。具名方法使用原始论文；当前框架行为应始终对照已安装版本重新检查。